Building data course — Week 7 — your own ontology: RDFS, OWL and SHACL

sparql-playground / lessons

Generated for print and offline reading

Week 7 — your own ontology: RDFS, OWL and SHACL

Five daily lessons on Burger Barn, a made-up burger restaurant. Each day has one short script, `dayNN_rdf.py`, that reads small hand-written Turtle files next to it in `lessons/unit4-rdf-by-hand/`. Run it with `uv run python lessons/course.py run N`. Every day ends with a box on how it maps to buildings (Brick, ASHRAE 223P, Project Haystack) and a

burger mini challenge; its answer is in the Try it section. Keep notes under the ignored `student-work/` folder.

Contents

- Day 31 — RDFS: classes, hierarchies, domain, range and entailment
- Day 32 — OWL properties: how things relate
- Day 33 — OWL classes: owl:Thing, disjointness, restrictions and the open world
- Day 34 — The menu graph: which menu items depend on the fryer?
- Day 35 — SHACL rules, and the bridge to a real building model

---

Day 31 — RDFS: classes, hierarchies, domain, range and entailment

Goal: build Burger Barn's class hierarchy with RDF Schema, find "all equipment" two ways, and see what an RDFS reasoner adds from `rdfs:domain` and `rdfs:range`.

Read

Week 6 used words like `bb:Fryer` and `bb:hasPoint` without saying what they mean. This week you write that down: your own **ontology**, in `burger_barn_ontology.ttl`. University courses teach it in layers, and so does this week: **RDFS** today, **OWL** on Days 32–33, **SHACL** on Day 35.

Two words you will see in papers and tools: the **TBox** ("terminology box") is the ontology, the *kinds* of things and properties; the **ABox** ("assertion box") is the facts about particular things. Here the TBox is `burger_barn_ontology.ttl` and the ABox is `burger_barn.ttl`. That's all the two words mean.

RDF Schema (RDFS 1.1) gives you four words that do most of the work:

RDFS	Means	Burger Barn
`rdfs:Class` / `owl:Class`	"this name is a kind of thing"	`bb:Fryer a owl:Class`
`rdfs:subClassOf`	"every X is also a Y"	`bb:Fryer rdfs:subClassOf bb:CookingEquipment`
`rdfs:domain`	"whatever has this property is	`bb:hasPoint rdfs:domain`

RDFS	Means	Burger Barn
	a …”	`bb:Equipment`
`rdfs:range`	“whatever this property points at is a …”	`bb:hasPoint rdfs:range bb:Point`

(The file says `owl:Class` rather than `rdfs:Class`. For today they mean the same; OWL tools prefer `owl:Class`.) At the very top sits `owl:Thing`: every class is a kind of Thing.

Now the important idea: **entailment**. The rules of RDFS say which extra triples *follow* from the ones you wrote. If `Fryer_1 a Fryer` and `Fryer subClassOf CookingEquipment`, then `Fryer_1 a CookingEquipment` follows, even though nobody wrote it. You can get those answers two ways: a **property path** in the query (`a/rdfs:subClassOf*`: “a type, then up the tree any number of steps”), or a **reasoner** that adds the entailed triples to the graph. `day31_rdf.py` does both, using `owlrl` with RDFS rules.

Run

```
uv run python lessons/course.py run 31
```

```
TBox (ontology) 134 triples, ABox (facts)
59 triples

Class tree (class -> parent):
   Burger -> MenuItem
   CookingEquipment -> Equipment
   DiningRoom -> Place
   DoorAlarm -> Point
   Drink -> MenuItem
   Equipment -> Thing
   FranchiseRegion -> Place
   Fryer -> CookingEquipment
   Grill -> CookingEquipment
   IceMachine -> Refrigeration
   Ingredient -> Thing
   Kitchen -> Place
   MenuItem -> Thing
   Place -> Thing
   Point -> Thing
   Refrigeration -> Equipment
   Restaurant -> Place
   SodaFountain -> Equipment
   TemperatureSensor -> Point
   WalkInCooler -> Refrigeration

From Fryer up: Fryer -> CookingEquipment -
> Equipment -> Thing
?thing a bb:Equipment:                  []
?thing a/rdfs:subClassOf* bb:Equipment:
['Fryer_1', 'Fryer_2', 'Grill_1',
'IceMachine_1', 'SodaFountain_1',
'WalkInCooler_1']

RDFS entailment: triples 194 -> 449
Types given to the untyped mixer and its
point: [('Mixer_1', 'Equipment'),
('Mixer_1_Speed', 'Point')]
```

`?thing a bb:Equipment` finds nothing: no triple says exactly that. The path finds all six. Then the surprise: a brand-new mixer, written with **no type at all**, only `bb:Mixer_1 bb:hasPoint bb:Mixer_1_Speed`, comes out of the reasoner typed `Equipment`, and its point typed `Point`. That's domain and range at work. They are **not checks**: RDFS never says "error, that isn't equipment". It concludes "then it must be equipment". If you want a check, that's SHACL (Day 35).

How this maps to buildings

> **Buildings box. Brick** is built exactly like this: `brick:AHU rdfs:subClassOf brick:HVAC_Equipment`, which is a subclass of `brick:Equipment`. So `?x a/rdfs:subClassOf* brick:Equipment` finds AHUs, VAVs and pumps in one query. **ASHRAE 223P** has its own class tree (`s223:Equipment`, `s223:Sensor`, ...), and **Haystack 4** defines its tags with "is" relationships (`ahu` is an `equip`) that export to RDF as `rdfs:subClassOf`. And watch domain and range in real models: a mistyped `brick:hasPoint` on a room makes a reasoner call the room "equipment".

Why it matters

A class tree turns “every fryer, grill, cooler and ice machine” into one word, `Equipment`. Entailment is how a query for that word finds them all, whether you write the path or let a reasoner add the triples.

Try it

1. Add `bb:Toaster` as a kind of `bb:CookingEquipment` and a `bb:Toaster_1` in the facts. Does it show up in the path query? In the plain one?
2. What does the reasoner conclude if you write `bb:Kitchen_2 bb:hasPoint bb:Kitchen_2_Temp`?
3. **Mini challenge (Burger Barn):** add `bb:Shake_1 bb:hasCalories 600` with no type, then run the RDFS reasoner. What is the shake now?

Answers

1. The path query, yes. The plain query, no (until you run the reasoner).
2. That `Kitchen_2` is `Equipment`, because `bb:hasPoint` has domain `bb:Equipment`. Probably not what you meant: domain and range infer, they don’t check.
3. **A `bb:MenuItem`**, because `bb:hasCalories` has domain `bb:MenuItem`. (The course’s tests

check this answer.)

Day 32 — OWL properties: how things relate

Goal: know the three kinds of OWL property, and the five property types that let a reasoner fill in triples you didn't write: sub-property, inverse, symmetric, transitive and functional. Then a quick look at property chains.

Read

OWL (OWL 2 Primer) adds more precise words on top of RDFS. Today is all about properties. Each one below has a burger example in the ontology, and what it means for buildings.

Three kinds of property.

OWL	Links a thing to…	Burger Barn	Logic?
`owl:ObjectProperty`	another **thing**	`bb:hasPoint`, `bb:partOf`	yes
`owl:DatatypeProperty`	a **value** (literal)	`bb:oilSetpointC 175.0`, `bb:hasCalories 320`	yes

OWL	Links a thing to…	Burger Barn	Logic?
`owl:AnnotationProperty`	a note **for people**	`rdfs:label`, `rdfs:comment`, `skos:definition`	**none**

Annotations carry no logic: a reasoner never infers anything from a label or a definition. They are for readers.

> **Buildings box.** `brick:hasPoint` is an object property (equipment → point). A setpoint's *value* is a datatype property (`brick:value 21.5`). `rdfs:label` and `skos:definition` are how **Brick** documents every class: annotations.

`rdfs:subPropertyOf`: one property is a special case of another. `bb:cookedOn` and `bb:storedIn` are both kinds of `bb:usesEquipment`. "Crispy chicken is cooked on Fryer_1" means "crispy chicken uses Fryer_1".

> **Buildings box. Brick**'s `brick:feedsAir` is a sub-property of `brick:feeds`. Ask for `brick:feeds` and a reasoner (or the path `?p rdfs:subPropertyOf* brick:feeds`) also finds the air connections.

`owl:inverseOf`**: the same link, read backwards.** `bb:usesEquipment` / `bb:isUsedBy`, and `bb:hasPoint` / `bb:isPointOf`. Write one direction; the other follows.

> **Buildings box. Brick** declares the pairs `brick:hasPoint` / `brick:isPointOf`, `brick:feeds` / `brick:isFedBy` and `brick:hasPart` / `brick:isPartOf`. **Haystack**'s `equipRef` on a point is the `isPointOf` direction.

`owl:SymmetricProperty`**: works both ways by definition.** If fries share a fryer with crispy chicken, crispy chicken shares a fryer with fries. Also `bb:adjacentTo` between rooms.

> **Buildings box.** "Room 101 is next to room 102" is symmetric. Core **Brick** has no adjacency relationship, so a model that needs one adds it in its own extension (Week 8 shows how) and declares it symmetric: one triple per pair instead of two.

`owl:TransitiveProperty`**: chains add up.** The kitchen is part of the restaurant, the restaurant is part of the North franchise region, so the kitchen is part of the region. Also `bb:contains`.

> **Buildings box.** A room is part of a floor, which is part of a building: **Brick**'s `brick:hasPart` / `brick:isPartOf`, which Brick's designers describe as transitive. And air flows AHU → VAV → zone through `brick:feeds`; "everything downstream of AHU_1" is `brick:feeds+` (Day 35).

`owl:FunctionalProperty`: at most one value. A menu item has exactly one calorie number; a fryer has one serial number. If the data gives two, OWL doesn't flag it: it concludes the two must be *the same* value, which for two different numbers means the data is inconsistent. In practice you check it with a query (`GROUP BY ... HAVING (COUNT(...) > 1)`, which the script does) or a SHACL `sh:maxCount 1` (Day 35).

> **Buildings box.** One `brick:hasUnit` per point, one serial number per controller: functional in meaning, checked with SHACL in practice.

Property chains, briefly. `owl:propertyChainAxiom ( bb:hasIngredient bb:usesEquipment )` defines `bb:needsEquipment`: a burger has a beef patty, the patty uses the grill, so the burger *needs* the grill. The

property path `bb:hasIngredient/bb:usesEquipment` asks the same thing without a reasoner.

`day32_rdf.py` lists the properties by kind and type, then asks six questions three ways: a plain query, a property path (no reasoner), and the plain query again after `owlrl` has added what the OWL rules entail.

Run

```
uv run python lessons/course.py run 32
```

```
ObjectProperty (16): adjacentTo, contains,
cookedOn, hasAlarm, hasIngredient,
hasPoint, includes, isPointOf, isUsedBy,
locatedIn, needsEquipment, partOf,
servesItem, sharesFryerWith, storedIn,
usesEquipment
DatatypeProperty (12): airSetpointC,
fryTime, hasCalories, isOpen,
lastInspected, oilSetpointC, openedOn,
opensAt, seats, serialNumber, specialty,
unit
AnnotationProperty (3): comment,
definition, label
TransitiveProperty: ['contains', 'partOf']
SymmetricProperty: ['adjacentTo',
'sharesFryerWith']
FunctionalProperty: ['hasCalories',
'serialNumber']
Menu items with two calorie numbers: []

Triples: before inference 241 | after 783

subPropertyOf: what does CrispyChicken
use?
  plain query, before inference: []
  property path, no inference:
['Fryer_1', 'WalkInCooler_1']
  plain query, after inference:
['Fryer_1', 'WalkInCooler_1']

inverseOf: who uses Fryer_1?
  plain query, before inference: []
  property path, no inference:
['CrispyChicken', 'Fries']
  plain query, after inference:
['CrispyChicken', 'Fries']
```

```
inverseOf: whose point is the oil temp?
  plain query, before inference: []
  property path, no inference:
['Fryer_1']
  plain query, after inference:
['Fryer_1']

symmetric: who shares a fryer with
CrispyChicken?
  plain query, before inference: []
  property path, no inference:   ['Fries']
  plain query, after inference:  ['Fries']

transitive: what is MainKitchen part of?
  plain query, before inference:
['BurgerBarn']
  property path, no inference:
['BurgerBarn', 'NorthRegion']
  plain query, after inference:
['BurgerBarn', 'NorthRegion']

chain: what equipment does ClassicBurger
need?
  plain query, before inference: []
  property path, no inference:
['Grill_1', 'WalkInCooler_1']
  plain query, after inference:
['Grill_1', 'WalkInCooler_1']
```

Every plain query is empty (or short) before inference and complete after, and the property path gets the same answer with no reasoner at all. The reasoner more than triples the graph to get there.

How this maps to buildings

> **Buildings box.** The boxes above, in one list:
>
> - **object / datatype / annotation:** `brick:hasPoint` / `brick:value` / the `skos:definition` on every **Brick** class
> - **sub-property:** `brick:feedsAir` is a kind of `brick:feeds`
> - **inverse:** `brick:hasPoint` / `brick:isPointOf`, `brick:feeds` / `brick:isFedBy`; **Haystack**'s `equipRef`
> - **symmetric:** room adjacency, in your own extension
> - **transitive:** `brick:hasPart` / `brick:isPartOf`
> - **functional:** one unit or serial number per point, checked with SHACL
>
> **ASHRAE 223P** uses the same OWL property machinery for its connections, and checks them with SHACL.

Why it matters

These five property types are most of what building ontologies use. Knowing them, you can read Brick's property definitions and

know which triples a tool will fill in for you, and which you must write.

Try it

1. Delete `owl:SymmetricProperty` from `bb:sharesFryerWith` and re-run. Which "after" answer changes?
2. Which is easier to explain to a colleague: `bb:MainKitchen bb:partOf+ ?x` or "run the reasoner first"?
3. **Mini challenge (Burger Barn):** with only a property path (no reasoner), list everything that is part of the North region, at any depth.

Answers

1. "who shares a fryer with CrispyChicken?" after inference becomes `[]`; the path still finds Fries, because `^` reads the triple backwards whatever the ontology says.
2. A matter of taste. The path is explicit and needs nothing extra; the reasoner keeps every query short.
3. **BurgerBarn, DiningRoom, MainKitchen**: `SELECT ?part WHERE { ?part bb:partOf+ bb:NorthRegion }`. (The course's tests check this answer.)

Day 33 — OWL classes: owl:Thing, disjointness, restrictions and the open world

Goal: see what OWL says about classes (everything is an `owl:Thing`, some classes can't overlap, a class can require something), and why OWL can't catch a missing value while SHACL can.

Read

Day 31 built the class tree with RDFS. OWL adds a few more class words. These are the ones you'll meet in building ontologies:

OWL	Means	Burger Barn
`owl:Thing`	the top class: everything is one	`bb:Equipment rdfs:subClassOf owl:Thing`
`owl:Class`	"this is a class" (an OWL tool's way of saying it)	`bb:Fryer a owl:Class`
`owl:disjointWith`	nothing can be in both classes	cooking equipment vs

OWL	Means	Burger Barn
		refrigeration, burger vs drink
`owl:Restriction` + `owl:someValuesFrom`	every member has *some* value of this kind	every fryer has some temperature point

A **restriction** is an unnamed class, written `[ ... ]` in Turtle. In `burger_barn_ontology.ttl`:

```
bb:Fryer rdfs:subClassOf [
    a owl:Restriction ;
    owl:onProperty bb:hasPoint ;
    owl:someValuesFrom
bb:TemperatureSensor
] .
```

Read it as “every fryer is one of the things that have some temperature point”. (`owl:cardinality`, `owl:minCardinality` and `owl:allValuesFrom` are its cousins, met in Week 8.)

Now the idea that surprises everyone: the **open-world assumption**. OWL assumes your data is *incomplete*: what isn’t written down might still be true. Fryer_2 has no temperature point in the data. Does OWL complain? No. It concludes “Fryer_2 has a temperature point we don’t know about yet”. **No missing value is ever an error to**

OWL. What OWL *can* catch is a contradiction: something typed with two disjoint classes can't exist in any world.

SHACL (Day 35) works the other way, **closed world**: what isn't in the graph isn't there. That's why building ontologies pair them: OWL/RDFS to say what terms mean, SHACL to check that a model is complete.

`day33_rdf.py` runs the OWL reasoner on the real data, then on a copy with one deliberate mistake (the ice machine typed as a grill too), then SHACL on the real data.

Run

```
uv run python lessons/course.py run 33
```

```
Triples: before inference 193 | after 669
Fryer_1 is a: ['CookingEquipment',
'Equipment', 'Fryer', 'Thing']
Things that are an owl:Thing after
inference: 14

OWL problems in the real data: [] <- open
world: nothing missing to OWL
OWL problems after typing IceMachine_1 as
a Grill too:
   Disjoint classes bb:CookingEquipment
and bb:Refrigeration have a common
individual bb:IceMachine_1
SHACL problems with Fryer_2 (closed
world): ['A fryer needs a temperature
point.', 'Oil setpoint must be one decimal
number.']
```

Three results, one lesson. After inference Fryer_1 is a Fryer, CookingEquipment, Equipment **and** owl:Thing. The real data gives OWL nothing to complain about, even with Fryer_2's missing point. The ice machine typed as a grill breaks `owl:disjointWith`, and owlrl reports it. SHACL, checking only what is written, finds Fryer_2's missing point (and its text setpoint).

How this maps to buildings

> **Buildings box. Brick** keeps `brick:Point`, `brick:Equipment` and `brick:Location` as separate top-level classes: a sensor is never an AHU.

> Declare classes like these disjoint (as Burger Barn does) and "I typed a sensor as an AHU" becomes a contradiction a reasoner catches. **Brick** and **ASHRAE 223P** then ship **SHACL shapes** for the closed-world checks ("a VAV must have a temperature point"), because OWL can't. That is why 223P leans so heavily on SHACL. **Haystack** has no reasoner: its tooling checks tags directly, which is closed-world too.

Why it matters

"The reasoner found no problems" does not mean "the model is complete". For FDD you need both: meanings you can reason with, and checks that the points your rules need are really there.

Try it

1. Add `bb:Cola a bb:Burger` to the menu file. Which disjointness does it break?
2. Would OWL complain if a menu item had **no** calorie number? Why not?
3. **Mini challenge (Burger Barn):** write a SPARQL query, no reasoner, that finds anything typed with two disjoint classes.

Test it with the ice machine typed as a grill.

Answers

1. `bb:Burger owl:disjointWith bb:Drink`: Cola is already a `bb:Drink`.
2. No. Open world: OWL assumes the number exists but isn't written down. Only a closed-world check (a query or a SHACL `sh:minCount 1`) can say it's missing.
3. It returns `IceMachine_1, CookingEquipment, Refrigeration`:

```
SELECT ?thing ?class1 ?class2 WHERE {
  ?class1 owl:disjointWith ?class2 .
  ?thing a/rdfs:subClassOf* ?class1 .
  ?thing a/rdfs:subClassOf* ?class2 .
}
```

   (The course's tests check this answer.)

Day 34 — The menu graph: which menu items depend on the fryer?

Goal: join three files into one graph and answer “what depends on what” questions with property paths.

Read

`burger_barn_menu.ttl` says what Burger Barn serves, what’s in each item, and which equipment cooks or stores each part. It uses the same names as `burger_barn.ttl` (`bb:Fryer_1`), so loading both files into one graph joins them. No keys, no join columns: **the same IRI is the same thing**.

The menu has layers: a combo *includes* items, an item *has ingredients*, an ingredient is *cooked on* a fryer. A property path follows all of them in one pattern:

```
?item
(bb:includes|bb:hasIngredient)*/bb:cookedOn
?fryer .
```

Path piece	Means
`bb:includes|bb:hasIngredient`	either property
`( ... )*`	zero or more steps of it
`/bb:cookedOn`	then exactly one `cookedOn` step

So it matches Fries (cooked on Fryer_1 directly), the chicken sandwich (has crispy chicken, cooked on Fryer_1) and the Barn combo (includes fries). `day34_rdf.py` asks three questions: which items need a fryer, what comes off the menu if Fryer_2 breaks, and which items need the walk-in cooler.

Run

```
uv run python lessons/course.py run 34
```

```
Menu items that need a fryer:
   BarnCombo needs Fryer_1
   ChickenSandwich needs Fryer_1
   Fries needs Fryer_1
   OnionRings needs Fryer_2
If Fryer_2 breaks, off the menu:
['OnionRings']
Items that need the walk-in cooler:
['BarnCombo', 'ChickenSandwich',
'ClassicBurger']
```

If Fryer_2 breaks, only the onion rings go. If the walk-in cooler fails, half the menu does. The Barn combo is in both lists through a chain of three links, and nobody wrote “the combo needs the cooler” anywhere.

How this maps to buildings

> **Buildings box.** Swap the words: “which zones lose cooling if chiller 1 trips?” is `?zone ^brick:feeds+ :Chiller_1`, the same “what depends on this” walk over **Brick**’s `brick:feeds`. **ASHRAE 223P** models the connections in even more detail (equipment, connection points and the connections between them), so the walk follows the actual pipes and ducts. FDD tools use exactly this to group alarms by their root cause.

Why it matters

Dependency questions are where graphs beat tables. In SQL each layer is another self-join; here it’s one `*`.

Try it

1. What comes off the menu if the grill breaks?

2. Add a “Fish sandwich” with a “Fish fillet” ingredient cooked on Fryer_2. Re-run: what changes?
3. **Mini challenge (Burger Barn):** how many calories is the Barn combo? (Each item has `bb:hasCalories`; the combo doesn’t.)

Answers

1. Swap `bb:Fryer_2` for `bb:Grill_1` in the “breaks” query: the classic burger and the Barn combo.

2. The fish sandwich appears with Fryer_2 in the first list and in “if Fryer_2 breaks”.

3. **1010** (540 + 320 + 150):

```
SELECT (SUM(?calories) AS ?total)
WHERE {
  bb:BarnCombo bb:includes ?item . ?
item bb:hasCalories ?calories
}
```

   (The course’s tests check this answer.)

Day 35 — SHACL rules, and the bridge to a real building model

Goal: write Burger Barn's rules in SHACL, read a validation report, then run the same kind of rule and the same Week 6 queries on a tiny building written with Brick names.

Read

SHACL, the Shapes Constraint Language (W3C SHACL), is the standard way to write closed-world checks: "every X must have …". Where OWL (Day 33) assumes missing facts exist somewhere, SHACL checks what is actually in the graph. The rules are themselves triples, in `burger_barn_shapes.ttl`. The words you need, all defined in the spec:

SHACL	Means	Burger Barn
`sh:NodeShape`	a rule about whole things	`bb:FryerShape`
`sh:targetClass`	which things to check (subclasses included)	every `bb:Fryer`; every `bb:Equipment`

SHACL	Means	Burger Barn
`sh:property [ sh:path ... ]`	a **property shape**: a rule about one property's values	`bb:oilSetpointC`
`sh:minCount` / `sh:maxCount`	at least / at most this many values	exactly one setpoint
`sh:datatype`	values must have this datatype	`xsd:decimal`, so `"175"` fails
`sh:class`	values must be things of this class	a `bb:TemperatureSensor`
`sh:pattern`	text must match this regular expression	serial numbers like `BB-FRY-0001`
`sh:minLength`	text at least this long	catches the blank label
`sh:severity`	how bad: `sh:Violation` (default), `sh:Warning`, `sh:Info`	missing labels are warnings
`sh:message`	what to tell the person	"A fryer needs a temperature point."
`sh:in ( ... )` / `sh:hasValue`	the value must be one of this	Day 38's named cook methods

SHACL	Means	Burger Barn
	list / exactly this	

`sh:targetClass` isn't the only way to pick what to check. A class can *also be* a node shape (`bb:Fryer a owl:Class, rdfs:Class, sh:NodeShape ; sh:property [ ... ]`), and then SHACL checks every instance of that class: an *implicit class target*. Brick and ASHRAE 223P ship most of their rules that way. Today's file uses `sh:targetClass`; Day 39 puts the two side by side and says when to pick each.

Reading a validation report. `pyshacl.validate(data, shacl_graph=rules)` returns `conforms` (True only if there are no results at all, warnings included), plus a **report graph**: one `sh:ValidationResult` per problem, with `sh:focusNode` (which thing), `sh:resultPath` (which property), `sh:value` (the bad value, if any), `sh:resultSeverity` and `sh:resultMessage`. Since the report is a graph, you read it with SPARQL, which is what `day35_rdf.py` does.

Then the bridge. `tiny_building.ttl` is a hand-written building with **Brick** names: an AHU feeding two VAVs, each feeding a zone, with points. It carries a handful of Brick's real `rdfs:subClassOf` facts (`brick:AHU` →

`brick:HVAC_Equipment` → `brick:Equipment`) instead of the whole Brick ontology. `tiny_building_shapes.ttl` is the fryer rule with new words: every VAV needs a temperature point.

Burger Barn	Tiny building (Brick)
`bb:Fryer` `rdfs:subClassOf` `bb:CookingEquipment`	`brick:VAV` `rdfs:subClassOf` `brick:Terminal_Unit`
`bb:hasPoint` / `bb:isPointOf`	`brick:hasPoint` / `brick:isPointOf`
`bb:contains+`	`brick:feeds+` (AHU → VAV → zone)
"every fryer needs a temperature point"	"every VAV needs a temperature point"

Run

```
uv run python lessons/course.py run 35
```

```
Restaurant follows its rules: False
   Fryer_2         Violation  A fryer
needs a temperature point.
   Fryer_2         Violation  Oil setpoint
must be one decimal number.
   Grill_1         Violation  Serial
number must look like BB-ABC-1234.
   IceMachine_1    Warning    Missing
label.
   WalkInCooler_1  Warning    Blank label.
Building follows its rules: False
   VAV_2           Violation  A VAV needs
a temperature point.

Equipment in the building: ['AHU_1',
'VAV_1', 'VAV_2']
Building equipment with a temperature
sensor: [('AHU_1', 'AHU_1_SAT'), ('VAV_1',
'VAV_1_ZoneTemp')]
Downstream of AHU_1 (brick:feeds+):
['VAV_1', 'VAV_2', 'Zone_1', 'Zone_2']
```

Five results for the restaurant: Fryer_2's missing point and its text setpoint, the grill's badly formed serial number, and two warnings for labels. One for the building: VAV_2 has a damper command but no temperature sensor. The last three lines are Week 6's queries with Brick words: same patterns, new names.

How this maps to buildings

> **Buildings box.** This *is* buildings. **Brick** is published as an OWL/RDFS ontology **plus** SHACL shapes; **ASHRAE 223P** defines most of its rules as SHACL, so a 223P model is "valid" when it passes the standard's shapes; and **Haystack 4** defines its tags as defs that export to RDF/Turtle, so the same SPARQL and SHACL tools work on Haystack data. OWL isn't "just a web thing": it's how these standards say what an AHU *is*, and SHACL is how they check that your model of one is complete.

Why it matters

An FDD rule is only as good as the model it reads. SHACL turns "the model should have…" into a report you can run on every model, every time, before any rule trusts it.

Week 7 further reading

- W3C, RDF Schema 1.1
- W3C, OWL 2 Web Ontology Language Primer (sections 4–5: classes, properties)
- W3C, Shapes Constraint Language (SHACL) (sections 2 and 4: shapes and

core constraints)

- Brick: relationships, ASHRAE 223P, Project Haystack: RDF
- pyshacl, owlrl

Try it

1. Give `bb:Fryer_2` a temperature point and the setpoint `175.0`. How many results are left?
2. Change `sh:severity sh:Warning` to `sh:Info` on the missing-label rule. Does `conforms` change?
3. **Mini challenge (Burger Barn):** write a shape that says every fryer must have a serial number (`sh:minCount 1`). Which fryer breaks it?

Answers

1. Three: the grill's serial number and the two label warnings.

2. No. Any result, even an `sh:Info`, makes `conforms` False. Read the severities to decide what blocks you.

3. **Fryer_2**:

```
bb:SerialShape a sh:NodeShape ;
    sh:targetClass bb:Fryer ;
    sh:property [ sh:path
bb:serialNumber ; sh:minCount 1 ;
                  sh:message "A fryer
needs a serial number." ] .
```

(The course's tests check this answer.)

Further reading — Week 7

The lessons first, then these for exact behavior.

- W3C: RDF Schema 1.1
- W3C: OWL 2 Web Ontology Language Primer
- W3C: Shapes Constraint Language (SHACL)
- W3C: SPARQL 1.1 property paths
- owlrl (OWL 2 RL reasoner for rdflib)
- pySHACL
- Brick: relationships
- ASHRAE 223P (open223.info)
- Project Haystack: RDF